

openGauss数据表创建与表约束

难度（最高三星）：★★★ 建议学时（2-4学时）：4学时

实验说明

学习地图

本项目实验案例集是实现一个学生成绩管理系统，共分为9个实验任务，按照难度等级分为初级、中级、高级，如图1所示。



图1 学生成绩管理系统实验

本次实验主要完成其中的任务2，主要是创建学生成绩管理系统的数据表以及对数据表进行相关约束。

实验环境准备

(1) openGauss的安装与部署：该部分请参照实验任务1【[openGauss表空间、模](#)

[式、权限管理](#)】进行安装与部署。

(2) 数据库创建：请参照实验任务1【[openGauss表空间、模式、权限管理](#)】，或直接[下载相关SQL语句](#)进行数据库表空间创建、数据库模式创建、用户权限管理。

实验任务

任务描述

本次实验共计四个子任务，主要是完成学生成绩管理系统的数据表的创建，使用SQL语句对数据表进行约束。整个实验会有详细的需求说明、实施步骤和关键代码。

学习目标

完成本任务的学习后，你应当能：

1. 学会数据表创建；
2. 能理解数据完整性；
3. 能掌握数据表约束。

任务准备

前置知识

本实验案例，需提前学习openGauss的基本概念、作用及基础操作。可通过如下途径进行学习：

- 数字教材：《[数据库原理与应用：基于openGauss](#)》
- 技术文档：[openGauss白皮书](#)

任务实施

本实验的具体操作可参见视频。

任务1：创建学生表并添加相应约束练习

训练要点：使用SQL语句创建表并添加约束。

需求说明：根据数据库设计，为学生成绩管理系统创建学生表Student，要求如下：

- (1) 包括字段student_no, student_name, sex, grade_id, phone, address, born_date, email, identity_card。
- (2) 设定 student_no为主键。
- (3) identity_card字段添加唯一约束。
- (4) address字段设置默认值为地址不详。
- (5) born_date添加检查约束，要求出生日期在1980年1月1日之后。
- (6) sex添加检查约束，限制只能输入 '男' 或 '女' 。

实现思路及关键代码：

```
CREATE TABLE student (  
    student_no INT PRIMARY KEY,  
    student_name VARCHAR(50) NOT NULL,  
    sex VARCHAR(3) NOT NULL CHECK (sex IN ('男', '女')),  
    grade_id INT NOT NULL,  
    phone VARCHAR(11) NULL,  
    address VARCHAR(255) DEFAULT '地址不详',  
    born_date TIMESTAMP NOT NULL CHECK (born_date >= '1980-01-01'),  
    email VARCHAR(50) NULL,  
    identity_card VARCHAR(18) UNIQUE NOT NULL  
);
```

任务2：使用SQL语句设计年级表并添加相应约束

训练要点：使用SQL语句创建表并添加约束。

需求说明：根据数据库设计，为学生成绩管理系统创建年级表Grade，要求如下：

- (1) 包括字段grade_id和grade_name。
- (2) 将grade_id设为主键，且自增。
- (3) grade_name字段设定为非空。

实现思路及关键代码：

```
CREATE TABLE grade (  
    grade_id SERIAL PRIMARY KEY,  
    grade_name VARCHAR(50) NOT NULL  
);
```

任务3：使用SQL语句设计课程表并添加相应约束

训练要点：使用SQL语句创建表并添加约束。

需求说明：根据数据库设计，为学生成绩管理系统创建课程表Subject，要求如下：

- (1) 创建 subject表，包括字段subject_no, subject_name, class_hour, grade_id。
- (2) 设定 subject_No为主键。
- (3) 添加 class_hour的检查约束，确保学时大于等于0。
- (4) 添加外键约束，将grade_id引用到grade表的grade_id字段上。

实现思路及关键代码：

```
CREATE TABLE Subject (  

```

```
Subject_No SERIAL PRIMARY KEY,  
Subject_Name VARCHAR(50) NOT NULL,  
Class_Hour INT NOT NULL CHECK (Class_Hour >= 0),  
Grade_Id INT NOT NULL,  
FOREIGN KEY (Grade_Id) REFERENCES grade (grade_id)  
);
```

任务4：使用SQL语句设计成绩表并添加相应约束

训练要点：使用SQL语句创建表并添加约束。

需求说明：根据数据库设计，为学生成绩管理系统创建成绩表Result，要求如下：

- (1) 包括字段student_no,subject_no, student_result, exam_date。
- (2) 将 (student_no, subject_no, exam_date) 设为复合主键。
- (3) exam_Date设置默认值为当前系统日期。
- (4) student_result添加检查约束，确保分数在0到100之间。
- (5) 添加外键约束，将student_no引用到student表的student_no字段上，将

subject_no引用到subject表的subject_no字段上。

实现思路及关键代码：

```
CREATE TABLE Result (  
    Student_No INT NOT NULL,  
    Subject_No INT NOT NULL,  
    Student_Result INT NOT NULL CHECK (Student_Result BETWEEN 0 AND 100),  
    Exam_Date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    PRIMARY KEY (Student_No, Subject_No, Exam_Date),  
    FOREIGN KEY (Student_No) REFERENCES student (student_no),  
    FOREIGN KEY (Subject_No) REFERENCES Subject (Subject_No)  
);
```

随堂测试

1. 数据库管理员为用户创建了一个存储市场有关信息的数据库，用户无法向数据库中输入数据，可能是因为（ ）
 - A. 数据库过多
 - B. 数据库的约束不起作用
 - C. 用户输入数据行太多
 - D. 还没有创建数据库表，因此无法输入数据

2. 假设Course表是主表，Student表是子表，需要建立它们的主外键关系，则（ ）
 - A. 在设计Course表的时候进入到关系的设计
 - B. 在设计Student表的时候进入到关系的设计
 - C. Student表主键和Course表外键的名称必须一样
 - D. Student表主键和Course表外键必须是自动增长数据类型

3. 主键用来实施（ ）
 - A. 实体完整性约束
 - B. 引用完整性约束
 - C. 域完整性约束
 - D. 自定义完整性约束

4. 电话号码应当采用()格式的数据类型来存储

A. 字符

B. 整数

C. 浮点数

D. Bit

5. 表Course和表Teachers建立了主外键关系，Course为主表，Teachers为子表，正确的选项是()

A. Teachers表中存在Course表的外键

B. Teachers表中存在外键

C. Course 表中存在外键

D. Course表中存在 Teachers 表的外键

6. 表Course中的列CourseID是标识列，属于自动增长数据类型，标识种子是2，标识递增量是3。首先插入三行数据，然后再删除一行数据，再向表中增加数据行的时候，标识值将是()

A. 5

B. 8

C. 11

D. 2